

Project Architecture of Microprocessors

Local AI Chatbot



Project by: Simedroni Andra

Coordinator teachers: Prof. Dr. Ing. Mircea Giurgiu

Ioana Maeva Carbunescu

Contents

1. Introduction.....	3
2. State of the art	3
3. Theoretical Fundamentals	3
4. Implementation	4
5. Experimental results.....	7
6. Future improvements	7
7. Conclusion	8
8. Bibliography	8

1. Introduction

The aim of the project is to provide a graphical user interface (GUI) chatbot application that integrates the *Langchain* library and a *Llama2* model. The chatbot maintains a conversational context, responds to user queries in natural language and is built with the *Tkinter* library for a user-friendly interface.

2. State of the art

The chatbot convers state-of-the-art advancements in natural language processing (NLP) via the the *Langchain* library and the *Llama2* model. This model enables dynamic and context-aware responses, making the chatbot suitable for various applications, such as education and general conversation. The advantage is that this chatbot runs locally, so it can be further on developed as a private tool which can be customizable for the needs of a company (for customer support, for example) or other applications.

3. Theoretical Fundamentals

The most important concept is represented by **LangChain**, which is a Python-based framework designed for building applications that integrate with large language models (LLMs). It simplifies the development of tools that utilize LLMs by providing abstractions and utilities for chaining tasks, managing memory, and interacting with APIs. LangChain has 4 important **features**:



- a) Prompt engineering – provides tools for creating and managing complex prompts.
- b) Chains – enables developers to combine multiple LLM calls and tasks into cohesive workflow.
- c) Memory: supports context management, allowing chatbots and applications to maintain conversation history.
- d) Agents: allows dynamic decision-making by combining LLMs with tools like search engines or calculators.



Another tool that I used for the realization of this project is **Ollama**, an advanced family of open-source LLMs developed by Meta AI. It is designed to compete with proprietary models like OpenAI's GPT and offers state-of-the-art performance in natural language, understanding and generation. It is completely free, runs locally and is available in different sizes, such as 7B, 13B and 70B parameters, each of them being suitable for varying computational resources. It is pretrained on a vast corpus of data, enabling general-purpose language capabilities and it can be fine-tuned for specific applications or domain-specific tasks.

The reason why I chose to specifically work with **Llama2** is the compatibility and performance of running the Ollama model on my laptop (which has 4 cores, 8 threads, base clock 3.1 GHz, turbo up to 4.4 GHz and 16 GB of RAM, with no GPU), as Llama2 requires 10-12 GB of RAM, being suitable for the performances of my processor.

The last, but not least library that I used is **Tinkter**, the standard GUI toolkit included with Python. It includes a wide variety of widgets, such as buttons, labels, text boxes and menus, suitable for desktop applications and tools for data visualization. It was a powerful tool for creating the GUI, such that the query given to the local AI chatbot is asked in a more user-friendly way.

4. Implementation

The implementation consists of the following steps:

a) Template Definition

```
template = """
Here is the conversation history:
{context}

Question: {question}

Answer:
"""
```

This string defines the format for the chatbot's interactions. It includes:

- A placeholder for the conversation history ({context}).
- A placeholder for the user's question ({question}).
- An area for model's answer.

b) Initialize the LLM

The OllamaLLM model is initialized and integrated with the prompt template through a chain, allowing the chatbot to use the defined template and the model together.

```
# Initialize the LLM
model = OllamaLLM(model="llama2")
prompt = ChatPromptTemplate.from_template(template)
chain = prompt | model # Chain the prompt with the model
```

c) Create the ChatBotApp Class

This class defines the chatbot's GUI and behavior as follows:

- The method `__init__` is used to define the main application window(or “root window”) that serves as the foundation of my GUI. It is created using the Tk class from Tkinter. After that, I defined the parameters of the GUI's window, as well as the area where the user will write the query.

```
def __init__(self, root):
    self.context = "" # Initialize an empty conversation context
    self.root = root
    self.root.title("Andra's AI ChatBot")

    # Chat display area
    self.chat_area = tk.Text(root, wrap="word", state="disabled")
    self.chat_area.pack(padx=10, pady=10, expand=True, fill="both")

    # User input area
    self.entry = tk.Entry(root)
    self.entry.pack(padx=10, pady=5, fill="x")
    self.entry.bind("<Return>", self.handle_conversation)
```

- The method `handle_conversation` generates an AI response using the Ollama model, and updates the chat interface. It handles the interaction between the user and the chatbot by reading the text entered by the user in the input field (`self.entry`), removing leading or trailing whitespace for faster computing. After that, it verifies if the input from the user is equal with the string “bye”, exiting the program. Contrary, it displays the input from the user on the GUI's window, appends the AI's response to the chat area (in case no exceptions are found), and updates the conversation history (`self.context`), to maintain a coherent conversation flow.

```
def handle_conversation(self, event):
    user_input = self.entry.get().strip() # Get and clean user input
    if not user_input:
        return # Ignore empty inputs

    if user_input.lower() == "bye":
        self.root.quit()
        return

    # Display user input
    self._append_to_chat(f"You: {user_input}")
    self.entry.delete(0, tk.END)

    # Generate AI response
    try:
        result = chain.invoke({"context": self.context, "question": user_input})
        ai_response = result.strip()
    except Exception as e:
        ai_response = f"Error: {e}"

    # Display AI response
    self._append_to_chat(f"AI: {ai_response}")

    # Update context
    self.context += f"\nUser: {user_input}\nAI: {ai_response}"
```

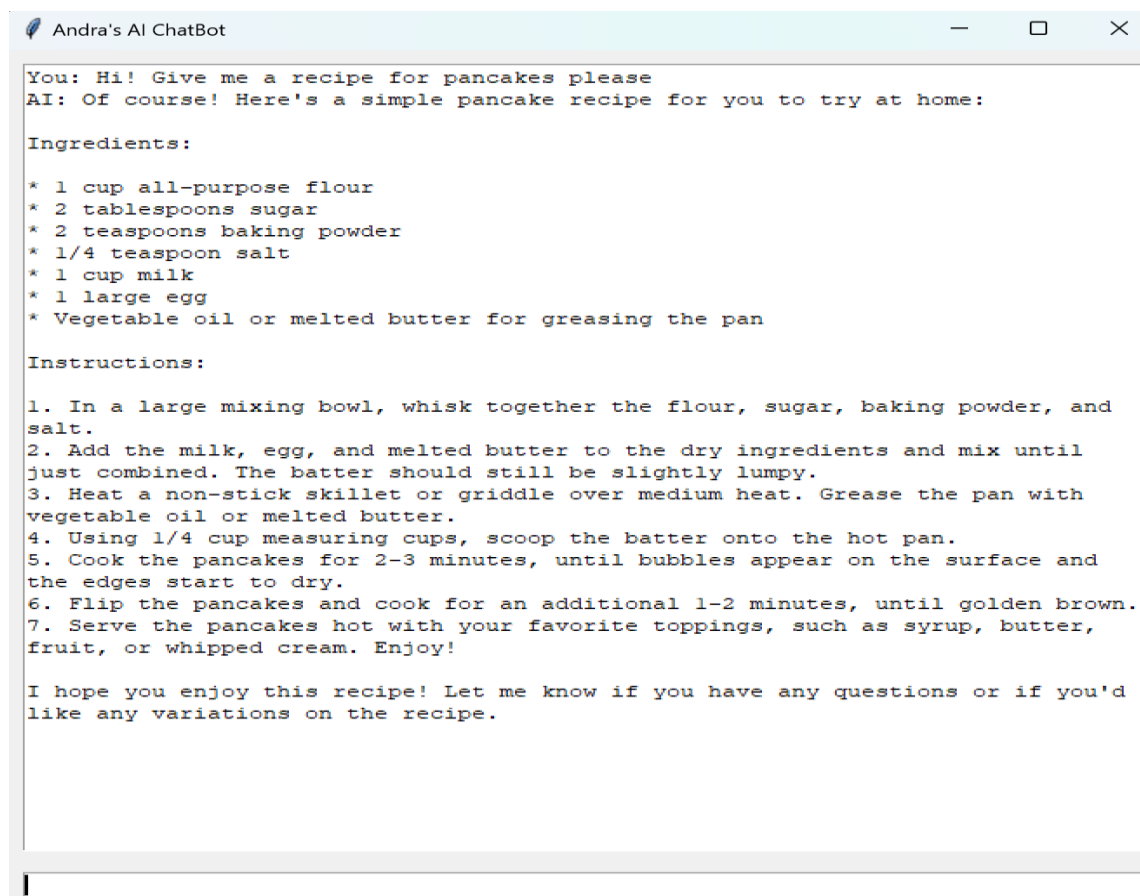
- The `_append_to_chat` method is frequently called by the `handle_conversation` method because it has the role of: editing Text widget, append the message to the chat display, prevents the user from modifying the chat history and automatically scrolls to the last message.

```
def _append_to_chat(self, message):
    """Append a message to the chat area."""
    self.chat_area.config(state="normal")
    self.chat_area.insert(tk.END, message + "\n")
    self.chat_area.config(state="disabled")
    self.chat_area.see(tk.END) # Auto-scroll to the latest message
```

- To run the application, I created a Tkinter window, I initialized the ChatBotApp and I started the Tkinter event loop.

```
root = tk.Tk()
app = ChatBotApp(root)
root.mainloop()
```

5. Experimental results



6. Future improvements

This chatbot can be improved by using RAG (retrieval augmented generation), meaning that this pretrained model can be trained continuously, by using external data (PDFs, photos etc.), such that it can become more specific. For example, if I want to ask it personal information about myself, it would not be able to give a correct answer, but if I'll give it some PDFs that contain information about myself, it will have a context for a right answer. This would be useful for a company, for example, which would make a personalized tool which can answer the questions of clients regarding different products or services.

7. Conclusion

This project is a perfect starting point for learning about AI and how it works, combining langchain for AI processing with tkinter for user interaction. It can be easily modified for personal needs and it runs locally, so it is a useful tool that can be furtherly improved.

8. Bibliography

<https://python.langchain.com/docs/introduction/>

<https://ollama.com/library/llama2>

https://www.youtube.com/run_ollama_locally

https://www.youtube.com/langchain_for_beginners